Hệ thống đặt lịch phòng khám nha khoa

Backend môn Kiến trúc phần mềm, Pha 1. **Java 17 · Spring Boot 3.5.16 · Spring Data JPA/Hibernate · Spring Security/JWT · SQLite.** Tài liệu tóm tắt phần trình bày 5–7 phút; bản A4 3 trang, dàn ý 6 phút 30 giây và hướng dẫn chi tiết.

1. Bài toán và phạm vi

Phòng khám cần tiếp nhận giờ khám khách mong muốn, bố trí bác sĩ và tránh phân công ca chồng giờ.

Vai trò	Nghiệp vụ
Khách hàng	Không cần tài khoản; xem thông tin/dịch vụ/bác sĩ, gửi yêu cầu khám, tra cứu hoặc hủy bằng mã.
Admin	Đăng nhập, phân công/đổi bác sĩ và giờ khám, quản lý hoặc vô hiệu hóa bác sĩ.
Bác sĩ	Đăng nhập, xem hồ sơ và lịch của mình; xác nhận, hoàn thành hoặc hủy theo trạng thái hợp lệ.

Phạm vi là REST backend; chưa có giao diện, thanh toán, hồ sơ bệnh án hay thông báo email/SMS.

2. Luồng chính và quy tắc dễ nhầm

Khách gửi **PENDING** → Admin phân bác sĩ/giờ thực tế **ASSIGNED** → bác sĩ xác nhận **CONFIRMED** → **COMPLETED**. ASSIGNED cũng có thể hoàn thành trực tiếp. PENDING/ASSIGNED/CONFIRMED có thể hủy sang CANCELLED; trạng thái kết thúc không mở lại.

• Giờ khách chọn là **mong muốn, chưa giữ chỗ**; khách không gửi doctorId. Lịch trống chỉ để tham khảo.

• Giờ khám ở tương lai, bắt đầu theo bước 30 phút và đủ thời lượng dịch vụ trong giờ mở cửa. Tra cứu ngày quá khứ trả 400.

• Cùng bác sĩ/ngày, trùng khi `start_cũ < end_mới && end_cũ > start_mới`. Ca 09:00–10:00 và 10:00–10:30 được nối tiếp.

• ASSIGNED/CONFIRMED/COMPLETED chiếm giờ; PENDING/CANCELLED không chặn. Service kiểm tra trước, **trigger SQLite** bảo vệ khi ghi; xung đột trả 409. `@Version` phát hiện cập nhật cùng bản ghi.

• Khách chỉ hủy trước giờ bắt đầu. Bác sĩ truy cập lịch người khác nhận 403. Không vô hiệu hóa bác sĩ còn lịch ASSIGNED/CONFIRMED.

3. Tổ chức mã nguồn và kỹ thuật

`HTTP/JSON → Security/JWT → Controller + DTO → Service → Repository → SQLite`

Vị trí	Trách nhiệm
`controller/`, `dto/`	26 API, ánh xạ HTTP, validation và DTO/mapper; không trả password hash.
`service/`	Appointment/Schedule/Doctor/Catalog/Auth: nghiệp vụ và giao dịch.
`repository/`, `entity/`	Spring Data JPA/ORM; 6 bảng accounts, doctors, clinics, dental_services, appointments, login_sessions.
`security/`, `exception/`, `config/`	JWT filter tập trung, principal, ánh xạ BusinessException sang HTTP, dữ liệu mẫu.
`src/main/resources/schema.sql`	Khóa ngoại, index, trigger chống chồng lịch; pool SQLite 1 kết nối.
`src/test/`, `.github/workflows/ci.yml`	API integration, ranh giới tầng/profile local; CI Java 17 và Docker smoke.

Service không import HTTP/Servlet/JPA/JDBC/Hibernate/Spring Data; `ArchitectureTest` kiểm tra ranh giới này. Spring `@Service`/`@Transactional` vẫn quản lý thành phần và giao dịch. Mật khẩu băm BCrypt; JWT 120 phút gắn phiên trong DB, nên đăng xuất/vô hiệu hóa tài khoản thu hồi quyền. Kiến trúc là một backend phân tầng, chưa phải microservices.

4. Chạy và xem đặc tả API

Yêu cầu JDK 17+ hoặc Docker. SQLite không cần database server.

```
./mvnw -B verify                # Windows: .\mvnw.cmd -B verify
./mvnw spring-boot:run           # Windows: .\mvnw.cmd spring-boot:run
# Hoặc:
docker compose up -d --build
```

API: http://localhost:8080; Swagger: http://localhost:8080/swagger-ui/index.html. Tài khoản giả lập: `admin / Admin123!`, `doctor1 / Doctor123!`, `doctor2 / Doctor123!`. Docker lưu SQLite trong volume; `docker compose down` giữ dữ liệu.

Đặc tả trong GitHub: OpenAPI 3.1 JSON · Preview Swagger UI · Ví dụ request. Preview dùng để đọc đặc tả; thử request bằng Swagger local khi backend đang chạy.

API gồm GET/POST/DELETE và PUT/PATCH: public 8, Auth 3, Doctor 5, Admin 10. Ví dụ GET `/api/admin/appointments` và POST `/api/admin/doctors` cần token ADMIN. Lỗi: 400 đầu vào sai, 401 chưa xác thực, 403 thiếu quyền, 404 không tìm thấy, 409 xung đột. Xuất lại đặc tả từ ứng dụng đang chạy: `python scripts/export_openapi.py`.

5. Kiểm thử và hiệu năng Kaggle CPU

19 test Java đạt (17 API, 1 kiến trúc, 1 profile local). Docker smoke kiểm tra luồng API, quyền, trùng giờ và dữ liệu sau restart. Test dùng database tạm.

Kịch bản mới: đúng **1 phiên Admin** ở mọi mức; **5/10/20 bác sĩ** có tài khoản riêng và **100/200/500 khách hàng** (tổng 106/211/521 phiên). Đây là tải giả lập, chưa phải cơ cấu nhân sự thực tế của một phòng khám.

Khách đọc, đặt–tra cứu; 20% yêu cầu mới hủy ngay, còn lại chờ xử lý. Admin đọc/phân công; bác sĩ đọc/xác nhận/hoàn thành. Tác vụ nghỉ 1–2,5 giây. Các phiên đăng nhập đầy đủ, làm nóng 15 giây, rồi reset thống kê và **giữ nguyên phiên** để đo 120 giây. Login/warm-up lưu riêng trong setup CSV.

Kaggle chạy thành công ngày 26/10/2026, mã nguồn `298d97f`. Xeon 2,20 GHz, 4 CPU logic/quota 4 CPU, RAM giới hạn 30 GiB; JDK 17, heap 512 MiB, SQLite pool 1, journal `delete`. Mỗi mức chạy một lần; thời gian đo thực tế 120,05–120,24 giây, gồm chờ request cuối hoàn tất.

Khách / BS / Admin	Request	RPS	TB (ms)	P95 (ms)	Lỗi
100 / 5 / 1	16.280	135,61	6,53	22	0%
200 / 10 / 1	32.477	270,50	10,26	32	0%
500 / 20 / 1	68.116	566,49	147,20	310	0%

P95 nghĩa là khoảng 95% request có độ trễ không vượt giá trị đó. Mức lớn nhất vẫn không lỗi nhưng độ trễ tăng rõ. **2.906 request đặt lịch chỉ đi cùng 41 lần phân công** trong cửa sổ đo: một Admin là giới hạn xử lý nghiệp vụ của kịch bản, nên RPS không phải số ca khám hoàn tất.

Bằng chứng: notebook đã chạy, có output · CSV tổng hợp · metadata/CSV theo mức và ZIP gốc · Kaggle v4. Notebook Kaggle đang riêng tư; bản thực thi và kết quả được lưu trong GitHub để người chấm đọc được.

Quy trình tái lập · Đối chiếu Pha 1. Kịch bản mới thay cả cơ cấu phiên, nghiệp vụ tạo tải và cách làm nóng; **không dùng số cũ 7/2/1 để kết luận tối ưu nhanh hơn**.

6. Vấn đề dễ thấy và hướng phát triển

Pool một kết nối SQLite, danh sách chưa phân trang và kiểm tra phiên mỗi request là các vị trí cần đo thêm. Một Admin phân công hữu hạn có thể làm hàng chờ PENDING tăng dù API còn nhanh. Chưa có migration schema, lịch nghỉ riêng từng bác sĩ hoặc đổi mật khẩu.

RPS/P95 chỉ áp dụng cho workload đã ghi; Locust và backend chia sẻ Kaggle CPU, chưa xác định tải tối đa hay số người dùng/ngày. Pha 2 cần đo CPU/thời gian chờ/truy vấn, rồi thử cải tiến và đo lặp trên cùng phần cứng, dữ liệu và kịch bản.

Nguồn: Yêu cầu môn học · Báo cáo trước khi đổi kịch bản tải.